

Module 02 - Text Preprocessing, TF-IDF, Word Embeddings, Dimensionality Reduction & Semantic Search Engine - Cosine Similarity

Objective

In this assignment, you will build the "eyes and ears" of a text-based robot. You will take messy, noisy text commands, clean them up, and convert them into mathematical vectors.

By the end of this lab, you will understand:

1. **Text Preprocessing:** How to clean raw data for AI.
 2. **TF-IDF:** How to identify "important" keywords based on frequency.
 3. **Word Embeddings:** How to use vector arithmetic to find semantic relationships (e.g., "Robot is to Battery as Human is to...?").
 4. Build a **Semantic Search** function so Bot-9 can understand intent, not just keywords.
-

New Learning Outcomes:

1. **Preprocessing:** Cleaning real-world dirty data.
 2. **TF-IDF vs. Embeddings:** Comparing keyword search vs. meaning search.
 3. **Visualization:** Seeing how words cluster in 2D space.
 4. **Cosine Similarity:** The algorithm used by Netflix, Google, and ChatGPT to find relevant data.
-

Submission Requirements

Submit a Jupyter Notebook (.ipynb) or python file. **You can use Gen AI tools to generate code and as well code comments and documentation.**

Scenario

You are the lead engineer for a warehouse robot named "**Bot-9.**"

Bot-9 receives instructions from tired humans who make typos, use weird capitalization, and include unnecessary punctuation. Your job is to:

1. Clean the logs.
2. Identify the key objects in the logs (using TF-IDF).
3. Help Bot-9 understand analogies (using Word2Vec) so it can generalize knowledge.

Instructions

Task 1: The Cleaner (Preprocessing)

Write a Python function `clean_text(text)` that takes a raw string and returns a normalized string.

- **Input:** "[ALERT]!!! Go to the... KITCHEN & grab the (Apple)."
- **Requirements:**
 1. Convert to lowercase.
 2. Remove special characters (brackets, punctuation, symbols like !, &, .). Keep only letters and spaces.
 3. Remove extra whitespace.
- **Expected Output:** "alert go to the kitchen grab the apple"

Task 2: The Keyword Finder (TF-IDF)

You have a list of 3 documents (commands). Use Scikit-Learn's `TfidfVectorizer` to convert them into a matrix.

- **Corpus:**
 1. "clean the kitchen"
 2. "clean the garage"
 3. "inventory the kitchen and the garage"
- **Goal:** Display a DataFrame showing the TF-IDF score for every word.
- **Question to Answer:** Why does the word "clean" have a lower score in Document 1 than the word "kitchen"? (Hint: Think about Inverse Document Frequency).

Task 3: Visualizing The Brain (Embeddings & Dimensionality Reduction)

Load a pre-trained Glove model using the `gensim` library (we will use `glove-wiki-gigaword-50` for speed).

- **Subtask A:** Find the mathematical similarity between "robot" and "machine".
- **Subtask B:** Perform Vector Arithmetic to solve this analogy:
 - King - Man + Woman = ?
- **Subtask C (Robotics Context):** Solve this analogy using code:
 - "Kitchen" is to "Chef" as "Hospital" is to ... ?

Using the GloVe embeddings:

1. Select a list of 10 words: ['king', 'queen', 'man', 'woman', 'robot', 'ai', 'computer', 'pizza', 'burger', 'food'].
2. Extract their vectors (50 dimensions).
3. Use **PCA (Principal Component Analysis)** from sklearn to squash these 50 dimensions down to **2 dimensions**.
4. Create a Scatter Plot.
 - o Goal: You should visually see "Food" words clustered together, "Royalty" words together, and "Tech" words together.

Task 4: The "Semantic Search" Engine (Cosine Similarity)

This is the most important task for modern AI.

1. Define a "Database" of robot commands:
 - o docs = ["Pick up the red ball", "Move to the kitchen", "Stop immediately", "Charge battery"]
2. Receive a User Query: "Grab the sphere"
3. **The Challenge:** The word "Grab" and "Sphere" do **not** exist in the database. A keyword search would fail.
4. **The Solution:**
 - o Convert the Query into a vector (by averaging the GloVe vectors of its words).
 - o Convert all Database commands into vectors (averaging their words).
 - o Calculate the **Cosine Similarity** between the Query Vector and every Command Vector.
 - o Return the command with the highest score.
- **Expected Output:** The system should identify that "Grab the sphere" is most similar to "Pick up the red ball".

----- Sample Python Code for starters created with
Gemini Pro 3 preview-----

Refer [Module 2 folder colab notebook](#)[Links to an external site.](#) and a sample solution below:

```
# =====  
# MODULE 02 ASSIGNMENT SOLUTION  
# =====
```

```

# --- PRE-REQUISITES ---
# Run this cell to install necessary libraries if running locally.
# In Colab, these are usually pre-installed.
!pip install scikit-learn pandas gensim numpy

import re
import pandas as pd
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
import gensim.downloader as api

print("Libraries imported successfully!")

# =====
# TASK 1: THE CLEANER
# =====
print("\n--- TASK 1: TEXT PREPROCESSING ---")

def clean_text(text):
    """
    Cleans raw text by lowercasing, removing special chars, and stripping whitespace.
    """

    # 1. Lowercase
    text = text.lower()

    # 2. Remove special characters using Regex
    # [^a-z\s] means "replace anything that is NOT a letter or a space with nothing"
    text = re.sub(r'[^a-z\s]', '', text)

    # 3. Remove extra whitespace (e.g., "hello  world" -> "hello world")
    text = re.sub(r'\s+', ' ', text).strip()

    return text

# Test Data
raw_input = "[ALERT]!!! Go to the... KITCHEN & grab the (Apple)."
cleaned_output = clean_text(raw_input)

print(f"Raw: {raw_input}")
print(f"Cleaned: {cleaned_output}")

# Check expectation
expected = "alert go to the kitchen grab the apple"
assert cleaned_output == expected, "Error: Cleaning logic is incorrect!"
print("Test Passed!")

```

```

# =====
# TASK 2: TF-IDF (KEYWORD EXTRACTION)
# =====
print("\n--- TASK 2: TF-IDF REPRESENTATION ---")

# The Corpus
documents = [
    "clean the kitchen",
    "clean the garage",
    "inventory the kitchen and the garage"
]

# Initialize Vectorizer
vectorizer = TfidfVectorizer()

# Fit and Transform (Learn vocab and convert to numbers)
tfidf_matrix = vectorizer.fit_transform(documents)

# Convert to DataFrame for readability
df_tfidf = pd.DataFrame(
    tfidf_matrix.toarray(),
    index=['Doc 1', 'Doc 2', 'Doc 3'],
    columns=vectorizer.get_feature_names_out()
)

print(df_tfidf.round(2))

print("\n--- CONCEPT QUESTION ANSWER ---")
print("""
Why is 'clean' score lower?
'Clean' appears in 2 out of 3 documents. Its 'Inverse Document Frequency' (IDF) is lower
because it is common. 'Kitchen' in Doc 1 appears in fewer documents overall (relative to
total terms),
making it more 'unique' and thus giving it a higher TF-IDF score.
""")

# =====
# TASK 3: EMBEDDINGS (SEMANTIC ALGEBRA)
# =====
print("\n--- TASK 3: WORD EMBEDDINGS ---")
print("Downloading Glove Model (approx 60MB)... please wait.")

```

```

# Load a pre-trained model (50-dimensional vectors)
# This downloads a model trained on Wikipedia
model = api.load("glove-wiki-gigaword-50")

# Subtask A: Similarity
sim_score = model.similarity("robot", "machine")
print(f"\nSimilarity between 'robot' and 'machine': {sim_score:.4f}")

# Subtask B: Classic Arithmetic
print("\nAnalogy: King - Man + Woman = ?")
result_b = model.most_similar(positive=['woman', 'king'], negative=['man'], topn=1)
print(f"Result: {result_b[0][0]} (Score: {result_b[0][1]:.2f})")

# Subtask C: Robotics Context
print("\nAnalogy: Chef is to Kitchen as Doctor is to... ?")
# Logic: Kitchen + Doctor - Chef = Hospital
result_c = model.most_similar(positive=['kitchen', 'doctor'], negative=['chef'], topn=1)
print(f"Result: {result_c[0][0]} (Score: {result_c[0][1]:.2f})")

# Bonus Experiment
print("\nBonus: Robot - Metal + Flesh = ?")
result_d = model.most_similar(positive=['robot', 'flesh'], negative=['metal'], topn=1)
print(f"Result: {result_d[0][0]}")

# --- PRE-REQUISITES ---
# !pip install scikit-learn pandas gensim numpy matplotlib

import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import PCA
from sklearn.metrics.pairwise import cosine_similarity
import gensim.downloader as api

# Load GloVe Model (Pre-trained vectors)
print("Downloading GloVe Model... (This may take a minute)")
model = api.load("glove-wiki-gigaword-50")
print("Model Loaded!")

# =====
# TASK 1: THE CLEANER (Standard)
# =====

```

```

def clean_text(text):
    text = text.lower()
    text = re.sub(r'[^a-z\s]', '', text)
    return re.sub(r'\s+', ' ', text).strip()

# =====
# TASK 3 continuation: VISUALIZING THE BRAIN (PCA)
# =====
print("\n--- TASK 3: VISUALIZATION ---")

words_to_plot = ['king', 'queen', 'man', 'woman', 'robot', 'ai', 'computer', 'pizza', 'burger', 'food']

# 1. Extract vectors
word_vectors = np.array([model[w] for w in words_to_plot])

# 2. Reduce dimensions from 50 -> 2
pca = PCA(n_components=2)
result = pca.fit_transform(word_vectors)

# 3. Plot
plt.figure(figsize=(8, 6))
plt.scatter(result[:, 0], result[:, 1], color='blue')

# Label the dots
for i, word in enumerate(words_to_plot):
    plt.annotate(word, xy=(result[i, 0], result[i, 1]), xytext=(5, 2),
                 textcoords='offset points', ha='right', va='bottom')

plt.title("Word Embedding Space (PCA Projection)")
plt.grid(True)
plt.show()

print("Observation: Notice how 'Pizza/Burger' are grouped, and 'King/Queen' are grouped.")

# =====
# TASK 4: SEMANTIC SEARCH ENGINE (RAG PREVIEW)
# =====
print("\n--- TASK 4: SEMANTIC SEARCH ENGINE ---")

# The Robot's Database
commands = [
    "pick up the red ball",
    "move to the kitchen",
    "stop immediately",

```

```

    "charge battery"
]

# The User's Ambiguous Query
user_query = "grab the sphere"
print(f"User Query: '{user_query}'")

# Helper Function: Sentence to Vector
# We create a sentence vector by averaging the vectors of its words
def get_sentence_vector(sentence, model):
    words = clean_text(sentence).split()
    # Filter out words not in the model's vocabulary
    valid_words = [word for word in words if word in model]

    if not valid_words:
        return np.zeros(50) # Return empty vector if no words found

    # Stack vectors and calculate mean
    vector_stack = np.array([model[word] for word in valid_words])
    return np.mean(vector_stack, axis=0)

# 1. Vectorize the Database
command_vectors = np.array([get_sentence_vector(cmd, model) for cmd in commands])

# 2. Vectorize the Query
query_vector = get_sentence_vector(user_query, model).reshape(1, -1)

# 3. Calculate Cosine Similarity
# Compare query against all 4 commands at once
scores = cosine_similarity(query_vector, command_vectors)[0]

# 4. Find the Winner
best_match_index = np.argmax(scores)
best_match_score = scores[best_match_index]
best_match_text = commands[best_match_index]

# Output Results
print("\n--- SEARCH RESULTS ---")
for i, cmd in enumerate(commands):
    print(f"Command: '{cmd}' | Similarity Score: {scores[i]:.4f}")

print(f"\n🏆 BEST MATCH: '{best_match_text}' ({best_match_score:.4f})")
print("Success! The robot understood that 'grab sphere' means 'pick up ball'.")

```